JPA, Part 5

Converters, events, object-level concurrency control.

Learning goals

1. Learn to use converters to make automatic transformations of values.
2. Learn the idea of JPA events.
3. Understand the fundamentals of application-level concurrency control in JPA.

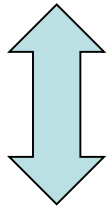
Converters

- Converters automatically transform an instance variable into a database-specific format, and vice versa.
- For example, a boolean Java variable can be converted into a database CHAR field that contains either Y or N.
- Converters are classes that implement the AttributeConverter interface.
- The interface defines two methods:
  - convertToDatabaseColumn()
  - convertToEntityAttribute()

Converters: the goal

Tili.java

```
private boolean lukittu;
```



tili (3r × 4c)

#	id	saldo	haltija_id	lukittu
1	**1**	127,05	1	e
2	**2**	6 352,5	2	e
3	**3**	0,32	1	k

Converters: an example

BooleanConverter.java

```java
@Converter(autoApply = false)  2 usages  olliv *
public class KyllaEiBooleanConverter
        implements AttributeConverter<Boolean, String> {
    @Override  no usages  olliv
    public String convertToDatabaseColumn(Boolean attribute) {
        // Java Boolean -> DB CHAR(1): 'k' tai 'e'
        if (attribute == null) {
            return "e"; // oletus
        }
        return attribute ? "k" : "e";
    }
    @Override  no usages  olliv *
    public Boolean convertToEntityAttribute(String dbData) {
        // DB CHAR(1) -> Java Boolean
        if (dbData == null) {
            return false; // oletus
        }
        if ("k".equalsIgnoreCase(dbData)) return true;
        if ("e".equalsIgnoreCase(dbData)) return false;
    }
```

Tili.java

```java
@Column(name = "lukittu", nullable = false, length = 1)
@Convert(converter = KyllaEiBooleanConverter.class)
private boolean lukittu;
```

Events

- Events are, in a way, the object-level counterpart of database triggers.
- Idea of the events:
  - An event listener class is attached to the target class. The event listener class reacts to state changes of target class objects.
  - Once a target class object's state changes in a way defined by the developer, a callback method in the listener class is automatically called.
- Event types:
  - @PrePersist and @PostPersist
  - @PreUpdate and @PostUpdate
  - @PreDelete and @PostDelete
  - @PostLoad
- Let's consider the use of events in an example where an event arises at each save.

Events: an example

Haltija.java

```
@Entity  vesavvo +1
@EntityListeners(HaltijaListener.class)
public class Haltija {
```

Halitjalistener.java

```
public class HaltijaListener {  2 usages  olliv
    @PostPersist  olliv
    public void afterPersist(Haltija h) {
        System.out.println("@PostPersist: Haltija tallennettu, id=" + h.getId());
    }
}
```

- Define a specific listener class that is registered with @EntityListeners annotation.
- In the listener class, the desired event handling methods (callback methods) are specified. The corresponding annotations specify the event types (in the example, @PostPersist).
  - Naming of the methods is free. There must not be a return value, and the paramater type must match the event source entity.
- The event handling methods should not use EntityManager to avoid unexpected chain reactions.

Concurrency control

- Concurrency control problems arise in situations where more than one transaction tries to operate on same data units simultaneously.
- DBMS has built-in concurrency control functionality.
  - Server of session specific transaction isolation level (e.g. READ_UNCOMMITTED or REPEATABLE_READ) defines how transactions see changes initiated by other transactions.
- Technically, each JPA transaction becomes a database transaction.

General Query Log

```
31 Query    SET autocommit=0
31 Query    select d1_0.deviceId,d1_0.description from DEVICE d1_0 where d1_0.deviceId=1
31 Query    select d1_0.deviceId,d1_0.description from DEVICE d1_0 where d1_0.deviceId=2
31 Query    commit
31 Query    SET autocommit=1
31 Quit
```

Application-level concurrency control

- Database-level concurrency control may not be sufficient for application level concurrency control.
  - Concurrency problems do not necessarily arise within any database transaction, but they may arise between applications
- Lost updates can happen, if two applications change the same value in the database.
  - E.g. an application can send a SELECT followed by an UPDATE. If another application updates the value in between, a lost update may occur.
- Application level concurrency control is not on by default when using JPA.
  - The corresponding functionality must be programmed into applications using the database.

Concurrency control problems

- The problems are same that in the DB level transactions:
  - Lost Update
  - Dirty Read
  - Incorrect Summary
  - Phantom Read
- Application-level concurrency control primarily targets lost updates (and more generally safe read–modify–write).

Techniques for concurrency control

- Three application-level lock strategies supported by JPA:
  1. Optimistic locking: uses a version attribute. On update/commit JPA includes the version in the WHERE clause; if the row was modified (version changed), an exception is thrown.
  2. Pessimistic locking: Pessimistic locking: database locks are acquired explicitly (e.g., SELECT ... FOR UPDATE for PESSIMISTIC_WRITE)
  3. No application-level locking: if no @Version is used and no pessimistic lock mode is requested, concurrent updates may overwrite each other (“last commit wins”)

Lock modes

- For the persistence operation, a lock mode (jakarta.persistence.LockModeType) is specified.

Lukkomoodi	Käytäntö
OPTIMISTIC	Optimistic locking
OPTIMISTIC_FORCE_INCREMENT	Optimistic locking, forces increment of the version number
PESSIMISTIC_READ	Pessimistinen locking, read lock
PESSIMISTIC_WRITE	Pessimistinen locking, write lock
PESSIMISTIC_FORCE_INCREMENT	Pessimistinen locking, write lock, forces increment of the version number (seldom used)
NONE	No locking

Optimistic locking

- Optimistic locking requires the version number to be included in the entity as an instance variable.
  - `long` data type is a good choice.
  - The version number is annotated with `@Version`.
- As the optimistic locking strategy is violated due to a contradiction in version numbers, an `OptimisticLockException` exception arises, and the application-level transaction is cancelled (rolled back).
- Handling this exception usually requires user input.
  - A fully automatic re-run of the transaction would *de facto* correspond to a situation of no locking at all. This would make, for example, a lost update possible.

Optimistic locking: example

Tili.java

```
@Version
@Column(name = "versio")
private long versio;
```

tili (3r × 5c)

#	id	saldo	haltija_id	lukittu	versio
1	1	127,05	1	e	0
2	2	6 352,5	2	e	0
3	3	50,32	1	k	10



tili (3r × 5c)

#	id	saldo	haltija_id	lukittu	versio
1	1	127,05	1	e	0
2	2	6 352,5	2	e	0
3	3	40,32	1	k	11

```
Hibernate:
    update
        tili
    set
        haltija_id=?,
        lukittu=?,
        saldo=?,
        versio=?
    where
        id=?
        and versio=?
```

- In the example, the version number grows by one, as the transaction is committed (or a flush() is carried out).
  - It generates an UPDATE statement whose WHERE clause verifies the version number.

Pessimistic locking

- In pessimistic locking, a transaction demands a lock and waits for it.
  - If the lock is not granted, a LockTimeOutException exception arises, and the application-level transaction is cancelled.
- Two types of locks:
  - PESSIMISTIC_READ is a read lock. Many transaction can have these locks for the same data unit.
  - PESSIMISTIC_WRITE is a write lock. It must appear alone for a data unit (no other read nor write locks may exist).